Comprehensive Non-Fatal Telemetry — §6.AC Surface Reference

Last updated: 2026-06-23 | Applies to: Android client (`:app`, `:core:analytics-firebase`) and `lava-api-go` service.

---

1. Architecture Overview

Lava implements §6.AC (Comprehensive Non-Fatal Telemetry Mandate) across two distinct artifact stacks. Each stack has its own implementation of the same conceptual contract: **every catch / error / fallback / unexpected-state path in production code MUST surface a telemetry event with enough context to triage the failure remotely.**

```
Android Client                                lava-api-go (Go service)
──────────────────────────────────────        ──────────────────────────────────────

AnalyticsTracker (interface)                  observability.RecordNonFatal()
  core/common/…/AnalyticsTracker.kt             lava-api-go/internal/observability/
  :33  recordNonFatal(throwable,ctx)              nonfatal.go :118
  :49  recordWarning(message,ctx)                 RecordWarning() :146

FirebaseAnalyticsTracker (impl)               OTLP structured log → Loki / Grafana
  core/analytics-firebase/…/                    (primary telemetry surface)
  FirebaseAnalyticsTracker.kt
  → Firebase Crashlytics non-fatal              Optional webhook bridge
    channel (mobile SDK only)                   (LAVA_API_NONFATAL_WEBHOOK_URL)
                                                → operator-configured HTTP collector
NoOpAnalyticsTracker (fallback)
  (used when Firebase clients null)
```

1.1 Android contract — `AnalyticsTracker`

File: `core/common/src/main/kotlin/lava/common/analytics/AnalyticsTracker.kt`

The project-wide interface declares two telemetry methods (lines 33, 49):

```
fun recordNonFatal(throwable: Throwable, context: Map<String,
        String> = emptyMap())
fun recordWarning(message: String, context: Map<String, String> =
        emptyMap())
```

`recordNonFatal` handles **throwable** exceptions caught by production code. `recordWarning` handles **non-throwable** unexpected situations — degraded paths, cache misses, mDNS-returned-zero, capability-declared-but-feature-null.

The `Params` object (lines 66–88) defines all canonical attribute keys in three tiers:

Tier	Keys	Source
§6.AC mandatory	`feature`, `module`, `operation`, `error_class`, `error_message`, `screen`	Lines 67–73
Domain-specific	`provider`, `query`, `category_id`, `topic_id`, `error`, `endpoint_kind`	Lines 76–81
HTTP-layer diagnostic	`http_status`, `request_url`, `http_method`, `base_url_host`	Lines 83–88

The HTTP-layer tier was added to enable structured triage of `ApiHttpException`-bearing failures without requiring Crashlytics dashboard access to the raw exception message.

1.2 Android implementation — `FirebaseAnalyticsTracker`

File: `core/analytics-firebase/src/main/kotlin/lava/analytics/firebase/FirebaseAnalyticsTracker.kt`

Key implementation details:

- **Constructor** (line ~20): `FirebaseAnalyticsTracker(analytics: FirebaseAnalytics?, crashlytics: FirebaseCrashlytics?)` — both clients are nullable. If both are null, the Hilt module installs `NoOpAnalyticsTracker` instead, so Firebase initialisation failure cannot crash production code.
- **`recordNonFatal`** (lines 51–66):
  1. Calls `isCancellationOrWraps()` (lines 76–85) — filters `CancellationException` and any exception whose cause chain (up to depth 32) contains one. Forensic anchor: Crashlytics issue `7df61fdba64f9928b067624d6db395ca` was 8 `JobCancellationException` events from `viewModelScope` teardown; those are structured-concurrency noise, not real failures.
  2. Sets each context entry as a Crashlytics custom key: `c.setCustomKey(key, value.take(MAX_VALUE_CHARS))` (line 63).
  3. Calls `c.recordException(throwable)` (line 64) to surface in the non-fatal feed.
- **`recordWarning`** (lines 96–105):
  1. Logs a breadcrumb: `c.log("WARN: ${message.take(MAX_VALUE_CHARS)} ctx=$context")` (line 101).
  2. Sets custom keys (line 102).
  3. Records `LavaNonFatalWarning(message)` (line 103) — a synthetic exception (line 129: `internal class LavaNonFatalWarning(message: String) :`

RuntimeException(message)) so non-throwable warnings surface in the **same** non-fatal feed as real exceptions.
- **MAX_VALUE_CHARS = 1024** (line 119): applied to every custom key value and to warning messages. Mirrors the Go side's maxValueChars = 1024 (nonfatal.go:91).

1.3 Go implementation — observability.RecordNonFatal

File: lava-api-go/internal/observability/nonfatal.go

```
RecordNonFatal(ctx, err, attrs)  :118
RecordWarning(ctx, message, attrs)  :146
```

Two effects per call:

1. **Always**: structured slog.WarnContext log (lines 133–138 for non-fatal, 148–153 for warning) via the OTLP pipeline → Loki/Grafana. The error, error_class, and every attrs key appear as queryable log fields.
2. **Optionally**: webhookForward() (line 139/153) to an operator-configured HTTP endpoint when LAVA_API_FIREBASE_CRASHLYTICS_ENABLED=true AND LAVA_API_NONFATAL_WEBHOOK_URL is set.

Cancellation filtering (lines 125–130): context.Canceled and context.DeadlineExceeded are silently skipped at DEBUG level — the Go-side symmetric of the Android CancellationException filter.

classOf(err) (line 238): walks the Unwrap() chain (max depth 32) to extract the deepest concrete type name via %T formatting. This was fixed in LVA-021: the prior fallback returned "error" for all fmt.Errorf-wrapped stdlib errors (*net.OpError, *url.Error, etc.), collapsing every backend HTTP failure into one Loki bucket.

---

2. Search-Failure Diagnosis Path (Priority Walkthrough)

This is the highest-value telemetry path: a user searches, a provider fails with HTTP 401 or connection-refused, and the operator needs to identify the failing provider + status code from a production Crashlytics or Loki record without a device attached.

Step-by-step (Android client)

Source: feature/search_result/src/main/kotlin/lava/search/result/SearchResultViewModel.kt, recordProviderFailure() at line 175.

When sdk.streamMultiSearch() (line 121) emits a MultiSearchEvent.ProviderFailure:

```
handleMultiSearchEvent()  :129
  → case ProviderFailure → recordProviderFailure(event)  :140
      builds baseContext  :176–182
        feature="search", operation="streamMultiSearch",
```

```
        screen="search_result", provider=event.providerId,
        error_message=event.reason
      if cause != null:  :183
        cause.rethrowIfCancellation()  :185  ← drops
CancellationException
        if cause is ApiHttpException:  :191
          extracts host-only from requestUrl  :192–198
          enriches context with HTTP keys:  :199–204
            http_status=cause.statusCode
            request_url=cause.requestUrl   ← query-stripped, no
credentials
            http_method=cause.httpMethod
            base_url_host=hostOnly
          analytics.recordNonFatal(cause, enrichedContext)  :208
        else:
          analytics.recordNonFatal(cause, baseContext)  :208
      else (no cause throwable):
        analytics.recordWarning(event.reason, baseContext)  :210
```

The result in Crashlytics: a non-fatal record whose custom keys include `provider=rutracker`, `http_status=401`, `request_url=https://api.vasic.digital/v1/rutracker/search`, `base_url_host=api.vasic.digital`. This is the information the operator needs to determine whether the failure is an auth-key expiry, a LAN routing issue, or a tracker outage.

Prior to this implementation the `ProviderFailure` branch was `-> Unit` (the `// §6.AC` comment in `handleMultiSearchEvent` line 142 is the original silent no-op). The operator-reported "release-only search failure" on 2026-06-22 was undiagnosable precisely because no context was captured.

The `ApiHttpException` shape

File: `core/tracker/client/src/main/kotlin/lava/tracker/client/ApiBackedTrackerClient.kt`, lines 48–54:

```kotlin
class ApiHttpException(
    val statusCode: Int,
    val requestUrl: String,   // query-stripped by
        stripQueryForTelemetry(:328)
    val httpMethod: String,
    val responseSnippet: String?,  // redacted+truncated to 200
        chars (:346)
    message: String,
) : IOException(message)
```

Thrown at lines 268–281 (`getString`), 283–296 (`getBytes`), 298–317 (`postJson`) whenever the upstream proxy returns a non-2xx status. Every throw already has query-stripped URL and credential-redacted response snippet — so the exception is safe to pass directly to `recordNonFatal`.

Parallel path on the Go side

File: `lava-api-go/internal/handlers/v1/handlers.go`, `writeProviderError()` at line 172.

All v1 handlers call `writeProviderError(c, err)` for provider-layer failures. The function's `default` branch (line 182) records unexpected errors:

```
observability.RecordNonFatal(c.Request.Context(), err,
        observability.NonFatalAttributes{
    observability.AttrFeature:   "provider",
    observability.AttrOperation: c.FullPath(),
    observability.AttrEndpoint:  c.FullPath(),
    observability.AttrTrackerID: currentProviderID(c),
    observability.AttrRequestID: requestID(c),
})
```

Sentinel errors (`ErrNotFound`, `ErrForbidden`, `ErrUnauthorized`, `ErrCircuitOpen`) are intentionally NOT recorded as non-fatals — they represent expected provider responses and would flood the feed. Only the `default` branch (unexpected provider implementation bugs, network failures, parsing errors) fires telemetry.

---

3. Mandatory Attributes and §6.H Redaction Rules

3.1 Mandatory attribute table

Both platforms share the same attribute vocabulary. The Go constants live in `nonfatal.go:69–87`; the Kotlin constants in `AnalyticsTracker.Params` at lines 66–88.

Attribute key	Kotlin constant	Go constant	Mandatory when	Example va
`feature`	`Params.FEATURE`	`AttrFeature`	Always	`"search", "auth"`
`module`	`Params.MODULE`	—	Android only	`"SearchRes`
`operation`	`Params.OPERATION`	`AttrOperation`	Always	`"streamMul "login"`
`error_class`	`Params.ERROR_CLASS`	`AttrErrorClass`	When known	`"ApiHttpEx "*net.OpEr`
`error_message`	`Params.ERROR_MESSAGE`	`AttrErrorMessage`	Always (truncated)	`"HTTP 401`
`screen`	`Params.SCREEN`	—	Android only	`"search_re`
`endpoint`	—	`AttrEndpoint`	Go only	`"/v1/rutra`
`request_id`	—	`AttrRequestID`	Go only	`"req-abc-1`
`tracker_id`	—	`AttrTrackerID`	Go only	`"rutracker`
`provider`	`Params.PROVIDER`	—		`"rutracker`

Attribute key	Kotlin constant	Go constant	Mandatory when	Example va
			Android, multi-search	
`http_status`	`Params.HTTP_STATUS`	—	Android, ApiHttpException	`"401"`
`request_url`	`Params.REQUEST_URL`	—	Android, ApiHttpException	`"https://…`
`http_method`	`Params.HTTP_METHOD`	—	Android, ApiHttpException	`"GET"`
`base_url_host`	`Params.BASE_URL_HOST`	—	Android, ApiHttpException	`"api.vasic`

3.2 §6.H redaction rules — what is NEVER logged

The following values are **categorically forbidden** from appearing in any telemetry attribute, log line, or webhook payload. Violations are a §6.H security incident.

Forbidden value class	Examples	Where the risk arises
Tracker login credentials	RuTracker username/ password, Kinozal API key	`LoginRequestDto` in `ApiBackedTrackerClient`
Session cookies / tokens	Tracker session cookie value, `sessionToken` field	`withAuth()` extension, `Auth-Token` header **value**
The `Lava-Auth` / `Auth-Token` header **value**	The tracker-specific base64 cookie string	Attached by `withAuth()` at `ApiBackedTrackerClient.kt:124`
Signed URLs / query-string tokens	Any `?token=`, `?key=`, `?auth=` query parameter	Stripped by `stripQueryForTelemetry()` at line 328
Firebase token (`LAVA_FIREBASE_TOKEN`)	The CI token for App Distribution	Never enters production code paths
Database connection string	Postgres DSN with password embedded	Go only — never passes through handler layer

Automatic redaction mechanisms:

- **Android** — `redactAndTruncate()` in `ApiBackedTrackerClient.kt:346` applies a regex over the response snippet before it reaches

`ApiHttpException.responseSnippet`. Pattern: `(?i)(token|key|cookie|auth|bearer|password|secret)([=:\s]+)\S+` → `$1$2[REDACTED]`.
- **Android** — `request_url` carried by `ApiHttpException` is the output of `stripQueryForTelemetry()` (line 328–334), which strips `?...` and everything after it.
- **Go** — `redactIfSensitive(key, value)` in `nonfatal.go:294` replaces the value with `"<redacted>"` for any attribute whose key substring-matches: `password`, `token`, `secret`, `api_key`, `apikey`, `cookie`, `authorization`, `hmac`, `pepper`.
- **Go** — The same redaction is applied to the webhook `attrs` copy at `nonfatal.go:196–200` before JSON encoding, so credentials cannot leak to the webhook collector either.

The `Auth-Token` header **name** (`"Auth-Token"`) is a §6.R-exempt wire-protocol constant at `ApiBackedTrackerClient.kt:356`. Its **value** (the session cookie) must never reach telemetry — only the header name is safe to reference in documentation.

---

4. Coverage Checker — `scripts/check-non-fatal-coverage.sh`

The §6.AC coverage checker is a bash script that scans Kotlin source under `core/` and `feature/` for `catch` blocks and flags those lacking a `recordNonFatal` / `recordWarning` call or an explicit `// no-telemetry: <reason>` opt-out comment.

Running the checker

```
# Advisory mode (default) — prints violations, exits 0
./scripts/check-non-fatal-coverage.sh

# Strict mode — exits 1 on any violation (used by scripts/ci.sh --
        full)
./scripts/check-non-fatal-coverage.sh --strict
```

What it scans

The checker looks for one of three coverage markers inside each `catch` block:

Marker	Meaning
`analytics.recordNonFatal(`	Real exception surfaced to telemetry
`analytics.recordWarning(`	Non-throwable warning surfaced to telemetry
`// no-telemetry:`	Explicit opt-out with a required reason
`rethrowIfCancellation()`	Followed by a recordNonFatal — CancellationException filtered first

Legitimate `// no-telemetry:` opt-outs

A `// no-telemetry:` opt-out is valid when the fallback is genuinely benign and recording it would create noise rather than signal. Two canonical examples already in the codebase:

```
// SearchResultViewModel.kt:141–143
// no-telemetry: a provider lacking TrackerCapability.SEARCH is a
// benign terminal state (skipped, not failed) per the
// MultiSearchEvent KDoc — not an error worth a non-fatal.
is MultiSearchEvent.ProviderUnsupported -> Unit

// General pattern for CancellationException paths
cause.rethrowIfCancellation()   // rethrows if cancellation;
        continues if not
analytics.recordNonFatal(cause,
        context)  // only reached for real errors
```

§6.AC-debt — mechanical enforcement status

The checker currently runs in **advisory mode** by default. `scripts/ci.sh --changed-only` does NOT yet hard-fail on coverage gaps. `scripts/ci.sh --full` runs `--strict`. The Go-side equivalent (`check-non-fatal-coverage.sh` for Go) is owed and tracked under §6.AC-debt. Reviewers manually verify every commit that adds a `try/catch` or `runCatching` block.

---

5. Operator Runbook

5.1 Android non-fatal dashboard — Firebase Crashlytics

Where: Firebase Console → Lava project → Crashlytics → Non-fatals tab.

Each non-fatal event surfaces with:

- The exception type (or `LavaNonFatalWarning` for warnings).
- The exception message.
- Crashlytics custom keys — these are the §6.AC attributes. Click any event → "Keys" tab.

Useful Crashlytics filter combinations:

Goal	Filter
All search failures	Non-fatals where `feature=search`
401 failures only	Non-fatals where `http_status=401`
Failures on a specific provider	Non-fatals where `provider=rutracker`
Failures hitting a specific backend	Non-fatals where `base_url_host=api.vasic.digital`

Closing a Crashlytics issue: per §6.O, mark "Resolved" in the Console ONLY after the fix commit lands AND `.lava-ci-evidence/crashlytics-resolved/<date>-<slug>.md` exists with: issue ID, stack-trace summary, root-cause analysis, fix commit SHA, and links to the regression test. Do not close-mark before the test coverage exists.

5.2 Go service dashboard — Loki / Grafana

Honest note: Firebase Crashlytics has **no public server-side non-fatal ingest API** — it is a mobile-SDK-only product. There is no POST endpoint that accepts non-fatal events from a Go server. The `nonfatal.go` package comment (lines 22–33) states this explicitly. The backend's primary telemetry surface is **Loki/Grafana** via the OTLP structured-log pipeline.

LogQL queries for common triage scenarios:

```
# All non-fatal events from the Go service
{service="lava-api-go"} |= "non-fatal event"

# Provider failures (unexpected errors in v1 handlers)
{service="lava-api-go"} |= "non-fatal event" | json |
feature="provider"

# Failures for a specific tracker
{service="lava-api-go"} |= "non-fatal event" | json |
tracker_id="rutracker"

# Warning events (fallbacks, capability mismatches)
{service="lava-api-go"} |= "warning event"

# Cross-reference with a trace: find the request_id in Tempo
{service="lava-api-go"} |= "non-fatal event" | json |
request_id="<id>"
```

5.3 Webhook bridge — optional forward to external collectors

When `LAVA_API_FIREBASE_CRASHLYTICS_ENABLED=true` AND `LAVA_API_NONFATAL_WEBHOOK_URL` is set (both from `.env`, per §6.R — never hardcoded), the Go service posts a best-effort JSON payload for each non-fatal/warning event (`nonfatal.go:184–233`).

JSON shape (`webhookBody` struct at line 164):

```
{
  "event": "non_fatal",
  "error_class": "*url.Error",
  "error_message": "HTTP 502 for /v1/rutracker/search…",
  "attrs": {
    "feature": "provider",
    "operation": "/v1/:provider/search",
    "tracker_id": "rutracker",
    "request_id": "req-abc-123"
  },
```

```
  "ts": "2026-06-23T14:05:00Z"
}
```

The webhook call:

- Runs in its own goroutine (fire-and-forget), **never blocks the user request path**.
- Has a 2-second timeout independent of the caller's context.
- Swallows all errors and recovers from panics (`nonfatal.go:219`).
- The `attrs` copy is credential-redacted before JSON encoding (`nonfatal.go:196–200`).

When the flag is set but the URL is empty, a one-time `WARN` log is emitted (`nonfatal.go:191`) and forwarding is silently disabled — the `sync.Once` at line 160 ensures this appears only once per process.

What the webhook enables: routing non-fatals to a Grafana Loki push endpoint, a custom Cloud Function, a Slack webhook, or — if the operator wires a Cloud Function — into Firebase Crashlytics from the server side. The bridge itself is transport-agnostic; the operator chooses the collector.

5.4 Checking for §6.H credential leaks in telemetry

If a non-fatal event's Crashlytics record or Loki log line contains what appears to be a credential, session token, or signed URL, file a §6.H incident immediately:

1. Record the event details in `.lava-ci-evidence/sixth-law-incidents/<date>-<slug>.json`.
2. Identify the call site where the attribute was constructed (the Crashlytics "Keys" tab shows exact key names; match to `AnalyticsTracker.Params.*` constants).
3. Verify `stripQueryForTelemetry()` and `redactAndTruncate()` are both applied before the value reaches `ApiHttpException`. If a new HTTP call site bypasses these helpers, that is the fix location.
4. On the Go side: verify `redactIfSensitive()` is applied. If a new attribute key name was introduced that doesn't contain one of the patterns in `sensitiveAttrPatterns` (line 98–108) but carries a sensitive value, add the pattern.
5. Rotate the exposed credential per §6.H clause 6.

File:Line Reference Index

Description	File	Lines
`AnalyticsTracker` interface — `recordNonFatal`	`core/common/…/AnalyticsTracker.kt`	33
`AnalyticsTracker` interface — `recordWarning`	`core/common/…/AnalyticsTracker.kt`	49
`Params` mandatory attribute constants	`core/common/…/AnalyticsTracker.kt`	66–73
`Params` HTTP-layer diagnostic constants		

Description	File	Lines
	core/common/…/AnalyticsTracker.kt	83–88
FirebaseAnalyticsTracker.recordNonFatal	core/analytics-firebase/…/FirebaseAnalyticsTracker.kt	51–66
isCancellationOrWraps() (CancellationException filter)	core/analytics-firebase/…/FirebaseAnalyticsTracker.kt	76–85
recordWarning → LavaNonFatalWarning synthetic exception	core/analytics-firebase/…/FirebaseAnalyticsTracker.kt	96–105, 129
MAX_VALUE_CHARS = 1024	core/analytics-firebase/…/FirebaseAnalyticsTracker.kt	119
ApiHttpException class definition	core/tracker/client/…/ApiBackedTrackerClient.kt	48–54
stripQueryForTelemetry()	core/tracker/client/…/ApiBackedTrackerClient.kt	328–334
redactAndTruncate()	core/tracker/client/…/ApiBackedTrackerClient.kt	346–350
AUTH_TOKEN_HEADER constant (name only, §6.R exempt)	core/tracker/client/…/ApiBackedTrackerClient.kt	356
recordProviderFailure() — search-401 diagnosis path	feature/search_result/…/SearchResultViewModel.kt	175–212
// no-telemetry: opt-out example (ProviderUnsupported)	feature/search_result/…/SearchResultViewModel.kt	141–144
Go RecordNonFatal()	lava-api-go/internal/observability/nonfatal.go	118–140
Go RecordWarning()	lava-api-go/internal/observability/nonfatal.go	146–154
Go sensitiveAttrPatterns (§6.H redaction list)	lava-api-go/internal/observability/nonfatal.go	98–108
Go webhookForward() — optional bridge	lava-api-go/internal/observability/nonfatal.go	184–233
Go classOf(err) — error type name extraction	lava-api-go/internal/observability/nonfatal.go	238–282
writeProviderError() — v1 handler non-fatal recording	lava-api-go/internal/handlers/v1/handlers.go	172–193